produces:

```
The middle one 2, the last one is 3
```

```
case [1, 2, 2]
in [*, value, ^value]
  puts "This has a pair of the same value, #{value}"
else
  puts "no pairs"
end
```

produces:

```
This has a pair of the same value, 2
```

Guard clauses in Patterns

pattern if *boolean_expression*

You can limit the ability of a pattern to match by putting a guard clause after it. The match only occurs if the pattern matches the target expression and the boolean expression in the guard clause is true. The guard clause may used values bound in the pattern.

Loops

while *boolean-expression* ‹do›
  body
end

This executes *body* zero or more times as long as *boolean-expression* is true.

until *boolean-expression* ‹do›
  body
end

This executes *body* zero or more times as long as *boolean-expression* is false.

In both forms, the do separates *boolean-expression* from the *body* and can be omitted when the body starts on a new line:

for ‹*name*›+ in *expression* ‹do›
  body
end

The for loop is executed as if it were the following each loop, except that local variables defined in the body of the for loop will be available outside the loop, and those defined within an iterator block will not.

expression.each do |‹*name*›+|
  body
end

loop, which iterates its associated block, is not a language construct—it is a method in module Kernel.

```
loop do
  print "Input: "
  break unless line = gets
  process(line)
end
```